

Batcher: A Mergeable Operator Algebra for Batch, Streaming and Inference, from One Core to a Cluster

Stephen Offer

Anyscale • stephen.offer@anyscale.com

Abstract

Every general-purpose data engine of the last fifteen years forked its stateful operators: once for one machine and once for a cluster, once for a bounded table and once for a stream, once for a relational query and once for a model. Each fork was the right answer to a real constraint. Their combined cost is why a team today runs four systems and maintains the glue.

We argue the fork was a choice rather than a necessity, and test that by building an engine that refuses it. Batcher admits a stateful operator only if it can be written once as a mergeable triple, $\text{partial} \rightarrow \text{combine} \rightarrow \text{finalize}$, in Rust over Apache Arrow; what does not qualify does not get added, and we publish the invoice. One core, ninety-six cores and a cluster then differ in *scheduling* and nothing else, so a small query pays nothing for distribution and a large one needs no rewrite. The same triple is the incremental form, so a bounded table is a stream that ends. An operator identical everywhere makes a measurement taken anywhere valid everywhere, which licenses planning from measured history instead of vendor constants. And decode, embedding and inference are expressions over the same batches, so a model is an operator inside the plan.

The evaluation tests the constraint before it tests the speed. All 22 TPC-H queries return identical results through the single-node and the distributed executor, and so do six injected worker-loss arms — a partitioning no planner chose. Then, on speed, the instrument is one controlled variable: DuckDB appears twice, differing only in whether it was allowed to ingest into its own compressed store. On identical Arrow input Batcher wins *every suite that comparison can run*, by 2.8 to 26 $\times$; given its own store back, DuckDB wins at ten times the data. Both are the same finding: what remains of the single-node incumbent’s advantage is a *storage format*, not an execution engine.

Four measurements the standard suites cannot see decide whether that is believable, and two are unflattering: Batcher buys its wall-clock win on TPC-H with 1.9 $\times$ the CPU on 2.4 $\times$ the cores, and shuffle replication makes recovery three times *more* expensive than the recompute it exists to avoid, while the shared operators drain a backlog at 3.2 $\times$ Spark Structured Streaming and a first-seen query converges to its steady state by the third execution against a control that isolates cache warming. Section 8 names the three axes not won, including the Join Order Benchmark [29], built to expose the estimation errors this design claims to correct, where Batcher remains 1.29 $\times$ behind.

1 Introduction

Every architecture encodes the world it was born into. The distributed data stack most companies still run was designed between 2009 and 2014, and designed well: the assumptions underneath it were true at the time. They have since stopped being true, and none of them gradually.

The machine inverted, and arrival became continuous. In 2012 a large server had 16 cores and 64 GB, so “the data will not fit on one machine” was simply a fact, and everything followed from it: a cluster you start before you ask a question, a scheduler between you and your data, serialization on every hop. A single cloud instance now carries 96 to 192 cores and terabytes of memory, and the tax stayed anyway, so a query that would take twelve milliseconds in-process still takes seconds through a distributed runtime. Change data capture and continuously landing object storage removed the nightly window in the same period, and the engines of the era answered by bolting a second execution model beside the first — two sets of operator semantics to hold in agreement, and a rewrite for any pipeline that crosses between them.

Nothing can be priced before the query runs, and most of the data is no longer columns. Classical optimizers assumed curated schemas, statistics refreshed on a schedule and queries that run for years. That world had `ANALYZE` jobs and DBAs; this one has object storage, schema drift, JSON columns, and operators that are opaque — a regular expression, a Python callback, an embedding model, an LLM call, none with a catalog entry, a selectivity or a cost anybody has measured. At the same time machine learning stopped consuming the finished dataset and started *producing* intermediate columns inside it, with a join or a filter on both sides; compute stopped being homogeneous, because a pipeline now decodes on CPU, runs a model on GPU and joins the result back on CPU; and the authoring language moved from SQL to Python, where the natural way to express a transformation is the one thing an optimizer cannot see into.

Each shift dissolved a boundary that used to be an architectural seam and is now a line one job routinely crosses: single-node and distributed, batch and streaming, structured and unstructured, query and model, CPU and GPU, SQL and Python. An engine built around either side of any of them must treat the other as foreign, and the bill arrives as glue.

This paper’s argument is that one constraint retires most of that bill at once, and that the constraint is a policy rather than an algebra. A stateful operator is admitted to Batcher only if it

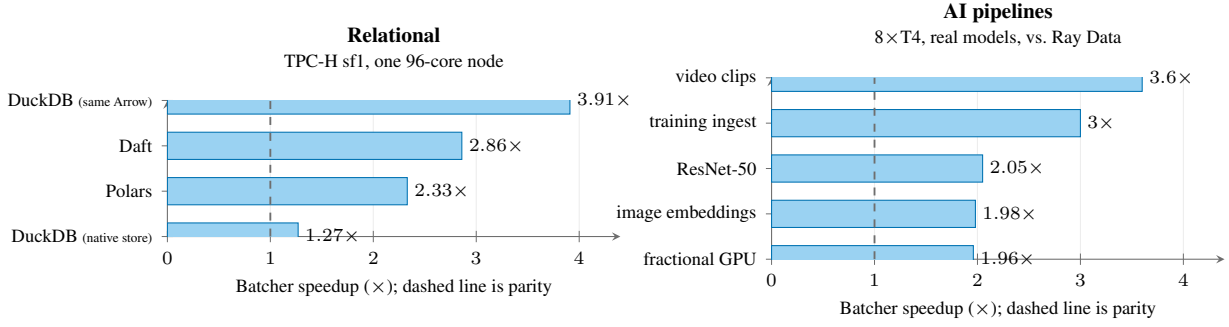


Figure 1: The claim in one picture: Batcher is not a relational engine that tolerates AI, nor an AI runtime that tolerates SQL. Left, a standard SQL benchmark against the fastest native engines (TPC-H sf1, all 22 queries, correctness-gated, geometric mean, one 96-core node); right, a runtime purpose-built for GPU pipelines, on its own workloads, with decode and inference treated as relational operators under the same optimizer. The bottom-left bar compares DuckDB’s compressed native store against Batcher reading raw Arrow — a storage engine *and* an execution engine against an execution engine alone — and inverts at ten times the data, which Section 7 attributes to storage rather than to kernels.

can be written once as a mergeable triple, $\text{partial} \rightarrow \text{combine} \rightarrow \text{finalize}$, over Arrow. The triple itself is old; making it the *condition of entry* is not, and it is what makes the partition count a free variable, the incremental form free, and a measurement taken on one core evidence about ninety-six. Breadth of this kind is normally bought by being second-best everywhere, and Figure 1 is the reason to think it was not bought that way here: on a standard SQL benchmark Batcher leads the fastest native engines, and on GPU pipelines it leads the runtime built for them, in the same binary and from the same plan.

1.1 Contributions

1. **Mergeability as an admission criterion, not a library interface.** The triple is old; making it the condition of entry to a single engine’s operator set is not. Summingbird and Beam unify the program or the specification *above* two or more engines, so the fork survives one level down; here there is nothing underneath to fall back to. Table 1 is the resulting vocabulary and Section 2.2 the invoice.
2. **One executor for batch and streaming, with a memory bound that is asserted and a driver cost that is disclosed.** Peak memory is flat in the input, the shared operators drain a backlog at $3.2\times$ Spark Structured Streaming, and a one-shot batch engine still beats the streaming driver by $13\times$ (Section 2.3, Section 7.5).
3. **Planning from measurement at three timescales, with a control.** A first-seen query costs $2.6\times$ its steady state and converges by the third execution, against a DuckDB control that isolates $1.1\times$ of that as cache warming — the measurement that separates a learning loop from a warm cache (Section 4, Section 7.6).
4. **Media, embedding and inference as ordinary relational expressions,** under one optimizer and one memory manager (Section 5), reachable from one lazy Python surface (Section 3).
5. **An evaluation designed to attribute, including where it is unflattering.** One controlled variable — whether DuckDB ingested into its own store — locates the residual

single-node gap in storage rather than execution. Beside it we report the three axes that did not favour us: CPU cost ($1.86\times$, Section 7.4), recovery under injected worker loss (Section 7.7), and the streaming driver’s $13\times$.

Reading the numbers. One convention runs through every table. A ratio written b/x is $\text{Batcher}_{ms}/x_{ms}$ for baseline x , so **below 1.00 means Batcher is faster**, and a suite figure is the geometric mean over its queries. Bar charts invert it to a speedup, where higher is better, and say so on the axis. Two DuckDB configurations appear throughout and are never merged: *native store* is DuckDB after an untimed ingest into its own compressed, dictionary-encoded, zone-mapped format, and *same Arrow* is DuckDB executing the identical zero-copy Arrow input Batcher is handed. The first is the harder bar and the one every published TPC-H result uses; the second is the only one that isolates execution from storage.

2 One algebra, every scale

The obstacle is easy to state. Ask an engine builder how a `GROUP BY` works and you get two answers: on one machine a hash table, on a cluster a shuffle, a partial pass and a reduce. The two share a name and a SQL semantics, and not a line of code — so two performance profiles, two sets of bugs, and eventually two different answers. A float key that hashes one way locally and another after a shuffle splits one group in two, while the single-node test suite stays green, because the single-node implementation is correct.

Batcher makes that split impossible. A stateful operator may exist only as a mergeable triple, and the three execution regimes vary only its scheduling (Figure 2). One core folds a stream of batches. Many cores partition and merge. Many machines partition, run the partial pass per node, ship pre-aggregated state, and combine in a tree. Formally, for any partitioning $\{p_k\}_{k=1}^n$ of an input multiset P ,

$$\text{finalize}\left(\bigoplus_{k=1}^n \text{partial}(p_k)\right) = \text{finalize}(\text{partial}(P)), \quad (1)$$

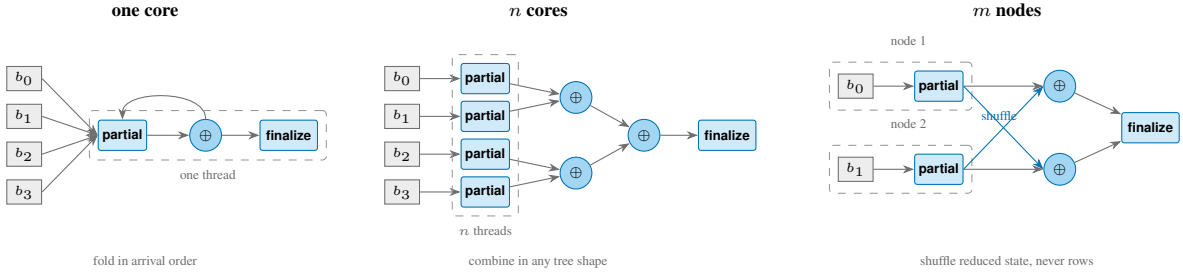


Figure 2: The same operator under all three schedulers. Only the arrangement of partial, $\oplus$ and finalize changes: one core folds each batch into a single accumulator, n cores fold per morsel and combine up a tree, and m nodes fold per node and exchange *already-reduced state* before combining. Associativity admits any tree shape, commutativity any arrival order, and the identity the empty partition, so the partition count is a free variable rather than a design decision. Because mappers publish state rather than rows, the bytes crossing the network on the right are bounded by the group count and not the input size, which is why per-node memory stays flat as the cluster widens.

Operator	partial	state S	$\oplus$
aggregate	hash to group states	group $\rightarrow$ acc.	merge by key
distinct	hash to key set	key set	set union
join (build)	hash to build side	hash table	merge by key
window	per-frame partials	frame $\rightarrow$ acc.	merge by frame
top- N	bounded heap	N heaviest rows	merge, re-truncate
<i>admitted under a weaker contract</i> (range-partitionable, not mergeable)			
sort, rank	ordered runs	sorted runs	boundary merge
<i>refused:</i> exact median/percentile, inequality join (distributed), shared global hash build, shrinking streaming state — Section 2.2			

Table 1: Batcher’s stateful operator vocabulary, and what each one must supply to be admitted. finalize is omitted: for every row it is the projection of S back to rows. This is the whole of the engine’s stateful surface — an operator absent from it is one the constraint refuses, and Section 2.2 is that invoice rather than a limitations note.

where $\oplus$ folds the per-partition states under the operator’s combine. This holds exactly when the operator factors into partial: $\mathcal{M}(R) \rightarrow S$, $\oplus: S \times S \rightarrow S$ and finalize: $S \rightarrow \mathcal{M}(R')$, with $\mathcal{M}(\cdot)$ a multiset, $(S, \oplus)$ a commutative monoid, and partial a monoid homomorphism from $(\mathcal{M}(R), \uplus)$ to $(S, \oplus)$. Each law earns its keep — associativity buys any tree shape, commutativity any arrival order, the identity the empty partition — and together they make the partition count a free variable. That is the whole of the claim: one core, many cores and many machines are three values of it. Equation (1) is a property test and not a comment, asserted over generated random partitionings against the single-pass result.

One qualification belongs with Equation (1) rather than in a footnote, because overstating it would make the invariant unfalsifiable. Row multisets, column names and column types are exact under any partitioning, but floating-point reductions are identical only *up to reassociation*: combine is associative in exact arithmetic, IEEE addition is not, and the partition count fixes the summation order. Neumaier compensation and Chan’s parallel formula for variance hold that error near the last bits; nothing can remove it while the partition count is free.

Two consequences follow. **There is no distribution tax on the small case**, because distribution is a scheduling decision made per query rather than a runtime you start; and **there is no rewrite on the way out**, because the operator that wins on one core is the one that runs on eight nodes.

2.1 The idea is old; the policy is not

Nothing above is a new algebra, and it would be dishonest to present it as one. Gray et al.’s algebraic aggregate [20] and MapReduce’s combiner [16] are the origin, Summingbird [9] made monoids the explicit unifying device, and the Dataflow model [1] generalized batch and streaming into one specification whose `CombineFn` has the shape of our triple. What we claim is the policy, and the difference is the word *only*. In each of those systems mergeability is an **opportunity**: a library interface an operator may implement to run better, alongside a general execution path for everything that does not, so the fork survives one level down — in the two engines Summingbird compiles onto, or in the runner Beam hands its specification to — and that is where the divergences and the second set of bugs live. Here mergeability is an **admission criterion** for the engine’s own operators: there is no general path to fall back to and no second runner underneath, so no second implementation can exist to drift from the first. That is a stronger claim and a more expensive one, and Section 2.2 is the invoice.

2.2 What the constraint refuses

A restriction is only interesting if something falls outside it, and Table 1’s last two blocks are the list. **Exact order statistics** are not algebraic aggregates, because the state is the input; Batcher admits the mergeable approximation (a KLL sketch [24]) and computes the exact form by falling back to a sort. **A global sort, a rank() over a total order and a recursive fixpoint** have no commutative combine, and no amount of design gives them one, so they are admitted under the weaker range-partitionable contract instead: the single algebra covers the stateful operators, not literally every operator.

Operators we can express but cannot yet distribute. The inequality join is the honest case, and it is a live cost rather than a historical one. IEJoin’s sweep over a sorted union has no mergeable decomposition without the block-pair pruning of [25], so today the distributed planner executes the operator whole. It satisfies single-node = distributed by not distributing, which honours the letter of the invariant and not its spirit (Section 8).

Algorithms we declined. A hash join that builds a shared global table across workers, and a streaming state backend with

tombstones, have no mergeable form we could find; the first is why the join build side is replicated rather than shared, and the second is why the incremental checkpoint below is restricted to operators whose state never shrinks. Neither was added.

The invoice is therefore real and not catastrophic: one operator class runs undistributed, one exact aggregate falls back to a sort, and two designs were rejected. We think that is a cheap price for deleting an entire axis of divergence, and the point of stating it is that a reader can now disagree with the trade rather than take our word that there was one.

2.3 Batch and streaming are one execution model

The constraint that settles the scale split settles this one too, because the mergeable triple *is* the incremental form. An aggregate that folds each batch through partial and combine never reads its whole input, which is exactly what a continuous query needs: state that updates per arrival and finalizes on demand. The operator a batch query uses and the operator a continuous query needs are not two implementations that happen to agree. They are one implementation, and the difference is whether anything ever calls finalize.

Batcher’s executor is therefore a streaming executor for *every* query. Operators pull small batches through a pipeline and materialize only at breakers, so peak memory is the breakers’ state plus one batch per worker: a constant, independent of how much data arrives. On Batcher’s own join-heavy q3/q4/q5 shape, same query and same engine, the streaming path holds 3.4 MB at one million rows and 3.3 MB at four million where the materializing path takes 198 MB and 794 MB. A query over a hundred terabytes of Parquet and the same query over a Kafka topic run the same operators under the same bound. The one place an incremental operator still costs something is the checkpoint, and the algebra pays there too: recording the *partial* each micro-batch folded in is linear where snapshotting the whole state is quadratic in the epoch count, and it is sound because a base plus every partial after it *is* the state. We do not claim Flink’s guarantees [12]: Batcher’s streaming is a micro-batch driver, and Section 8 states exactly which guarantees that fore-closes.

3 The programming interface

The historical bargain was that SQL is restrictive and therefore optimizable, while a user-defined function is expressive and therefore opaque. That bargain fails when the practitioner writes Python by default, because the natural way to express a transformation becomes a per-row callback the optimizer cannot reorder or push a predicate through. Batcher’s answer has three parts, in order of preference, and Figure 3 exercises all three.

First, make the UDF unnecessary. The public surface is a Python *expression* API — `col("x") * col("y"), .str.contains(...), .list.cosine_similarity(...), .image.to_tensor(...),`

```
import batcher as bt

frames = bt.read.images("s3://clips/**/*.*.jpg")
cat     = bt.read.parquet("s3://catalog/")

hot = (frames
      .filter(bt.col("width") >= 640)           # below the decode
      .with_columns(t=bt.col("image")
                    .image.to_tensor(224, 224))
      .ml.infer("resnet50", column="t",
                output_column="label")         # a GPU operator
      .join(cat, on="sku")
      .group_by("label")                       # mergeable
      .agg(n=bt.col("sku").count()))

hot.collect(distributed="auto")                # the only scale knob
```

Figure 3: A Batcher program. Decode, inference, join and aggregation are nodes in one logical plan, so the optimizer sees the filter and the decode together and runs the cheaper one first; nothing here is a user-defined function, and Python never touches a row. The last line is the only place scale appears: `distributed` changes which schedulers of Figure 2 run, not which operators and not the answer.

`.audio.mel_spectrogram(...)` — and every one lowers to the same `bc_expr::Expr` the SQL front-end produces, so it is optimized, JIT-eligible and executed in Rust. That is what makes the filter in Figure 3 movable below the decode: both are plan nodes with costs, not opaque calls. Second, when a callback is genuinely required, make it batch-first: a Batcher UDF receives a whole Arrow `RecordBatch` and returns one, and a class-based UDF keeps a loaded model or a connection across batches and executions, which is what makes session-warm GPU pools possible. Third, keep SQL, which parses to the same `LogicalPlan` and runs the same rules and kernels. Neither surface wraps the other, because both lower to one plan.

A `Dataset` is a lazy, immutable handle to that plan: every operation returns a new one and nothing executes until a terminal call, which is what lets the scale decision sit on the last line rather than in the constructor. It is the API-level statement of Section 2: the program does not know how many machines will run it.

4 Planning from measurement

An engine that cannot price its data or its operators before running can keep inventing numbers, which is what a static cost model does, or it can measure itself and plan from what it saw. Measuring itself is only sound because of the first pillar: because the same operator runs everywhere, **a measurement taken anywhere is valid everywhere**, so a cardinality observed on one core is evidence about the same operator on ninety-six. Refusing to fork the operator is what licenses the engine to plan from its own history. Every other engine below ships its cost model as constants, chooses join strategy by heuristic, and sizes memory from a plan estimate — constants tuned once, on hardware nobody here is running.

A single rule keeps this safe. **Every learned value is result-invariant.** Each bandit arm emits the same relation, a threshold picks between equivalent algorithms, a partition count only shards. Learning changes how much memory a query reserves, when it spills, how wide a batch is, and which of several equiv-

alent algorithms runs. It can never change the answer, and that is property-tested: a cold store reproduces the untuned engine byte for byte, so the worst a stale value can do is cost throughput. This is a deliberately different bet from learned optimizers that predict plans [26, 31, 32], which change which plan runs in ways that are hard to bound. A weaker signal buys a far stronger safety property, and that is what lets the feature ship on by default.

Three clocks, not one. Adaptive query processing usually means one thing: re-plan at a stage boundary. Batcher does that too, and it is the least interesting of the three places it adapts.

Two of the clocks run *inside* an operator, at microsecond grain, and both replace a number a static model would have to invent with one the operator just observed. A conjunctive filter orders its predicates by measured time-per-row over fraction-removed rather than by static cost, which matters because cost is not selectivity, and two predicates of identical cost where only one is selective is a case a static model cannot fix even in principle ($1.86\times$ on the adversarial shape). A parallel aggregate chooses between per-morsel hash tables and pre-partitioning by key from the reduction a sample of morsels actually achieved rather than from a distinct-count estimate ($3.7\times$ when grouping reduces nothing). The second matters architecturally rather than numerically: pre-partitioning *is* the right-hand panel of Figure 2 executed across cores instead of machines, so the high-cardinality single-node aggregate is the distributed algorithm chosen at runtime. Refusing to fork the operator did not only delete a class of bug; it handed the single-node executor a second strategy for free.

The outer two clocks are familiar. At a pipeline breaker the engine re-plans the remainder on real cardinalities — the same mechanism and granularity as Spark AQE [42], and we claim no finer grain, though it runs single-node where AQE needs shuffle stages. Across executions the engine accumulates sketches, refits cost coefficients and converges bandits, which Section 7.6 measures. Each clock reads only what has already been measured, so none is a prediction.

5 One engine across data shapes, model tiers and devices

The second fragmentation is a grid rather than a line: engines specialize by the *shape* of the data and by the *tier* of the model, the two axes fragment independently, and a team lands in the cross product. DuckDB and Polars are superb over structured columns and have no image or audio surface at all; Ray Data and Daft handle multimodal data well, but Ray Data carries no relational optimizer and Daft’s is rule- only, with no cost model to weigh a decode against the join above it; Spark’s multimodal story is a UDF.

Batcher registers 69 source formats and exposes 197 public ML entry points, from generalized linear models through GPU inference to LLM batch generation. What matters is less the size of those numbers than that a single plan spans them, because preparing data for a model is not a different kind of

work from querying it. Image decode, crop and normalization to a tensor are Arrow kernels in the data plane, not a Python UDF, and so are audio decode, resampling, mel-spectrogram and MFCC — the last two matched to `torchaudio` within 10^{-6} and, across the comparator set of Section 7, without an in-engine equivalent anywhere else. So are vector distances and embedding pooling. Two measurements bound what that is worth on the axis both sides run natively: against Ray Data’s `read_images` over the same corpus Batcher decodes $6.4\text{--}6.6\times$ faster, entirely the difference between an engine expression and a per-batch Python UDF, and against Daft on the shared ingest path it leads by $1.9\times$, rising to $5.7\times$ on a pass Daft expresses as a per-row callback.

One scheduler across CPU and GPU. Run a chain of unlike costs — I/O, then CPU decode, then a forward pass, then a CPU join — inside one worker and the accelerator idles through every decode; the usual remedy is for the user to hand-tune a ratio of CPU workers to GPU workers, per stage. Batcher treats it as a scheduling problem the engine owns: a plan carrying a resource- class boundary is split there, a pool of CPU producers decodes and preprocesses, a pool of GPU consumers runs the model, and Arrow batches move between them over the same credit-bounded Arrow Flight [44] transport the shuffle uses, so a slow consumer applies backpressure instead of accumulating. Both stages run the identical sub-plan through the ordinary executor; only the placement differs, which is what keeps the split from becoming a second semantics.

Ray Data’s streaming batch model [30] attacks the same problem and is the work to compare against rather than the baseline to beat. Making the *partition* the unit of execution buys it elasticity and partition-level fault tolerance, neither of which Batcher has. What Batcher buys instead is what that model does not attempt: both stages are the *same relational plan* under the *same cost model* as the joins around them, so the optimizer can push a filter below a decode because it sees both. The honest summary is a trade rather than a win, and Section 7.7 measures the half we are weaker on.

6 System

Batcher is a Python control plane over a Rust data plane, with Arrow `RecordBatch` [43] the only columnar contract at every internal boundary and across the FFI. Python builds a plan, optimizes it and ships it down as JSON; data returns zero-copy through the Arrow C Data Interface, and Python never iterates a row. That boundary is what makes a Python surface affordable: the control plane’s cost is per *query*, so seven phases of rules iterated to a fixpoint cost single-digit milliseconds because nothing in them is per-row (Section 7.4).

Three structural rules do the load-bearing work, and each exists because its violation is silent. *The subsystems may not import one another.* Kyber decides, Carbonite protects, Core measures, and a lint contract forbids any edge between them, which reads as bureaucratic until you see what it prevents: an optimizer pass that starts collecting its own telemetry, or an executor path that starts making optimization decisions, compiles

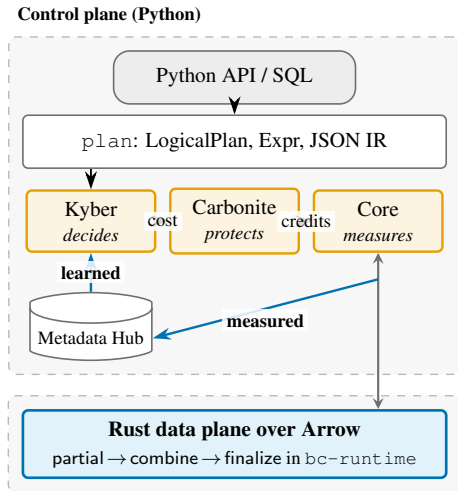


Figure 4: The contract loop. Python builds and optimizes a plan and ships it down as JSON IR; all per-row work happens in Rust over Arrow and comes back zero-copy, which is the grey double arrow. *Core measures*, *Kyber decides*, *Carbonite protects*, and the three cannot import one another, enforced on every commit. The blue path is the one the other engines do not have: what execution measured is what the next plan is built from, so a query gets cheaper the second time it runs.

and passes every test while quietly corrupting the feedback loop the design depends on. The only sound edge is the one through the Metadata Hub in Figure 4, and it is directed — measurements flow out of execution into planning, never back. *One expression type serves every execution tier*. An interpreter is the correctness oracle and a Cranelift [11] compiled fast path [36] consumes the *same* expression, so it must be bit- for-bit identical on the subset it supports and fall back silently otherwise; a divergence — one tier wrapping an integer overflow where the other raises — is invisible in any single result, because which tier ran depends only on whether the expression happened to compile. *The crate dependency graph points one way*, from the Arrow layer up through one expression type and one relational plan type to the interpreter, so a new backend cannot introduce a second definition of what a plan node means. Underneath all three the executor pulls small batches through linear runs of operators and materializes only at pipeline breakers [28] — which is exactly where the middle clock of Section 4 re-plans.

Kyber, the optimizer. Kyber is a fixed sequence of phases over the plan, and the architecturally interesting property is not the rule count (726 at the commit these results were taken on) but the *ordering of evidence*. Estimation starts from Selinger-style selectivities [40] and is superseded, in a fixed order, by exact file footer metadata, then by sketches built during execution (HyperLogLog [18], KLL [24], Count- Min [14]), then by measured cardinalities filed under a plan signature, with a bandit [5] over result-equivalent physical strategies on top. Every sketch is mergeable under a fixed seed, so one built per partition merges identically to one built on a single node — Section 2’s algebra applied to statistics, and the reason a distributed run’s evidence is admissible to a single-node plan. Every number also carries *where it came from*, because two rewrites can

corrupt a result rather than merely slow one and are gated on exact provenance, while everything else may consume an estimate whose worst outcome is a slower plan. A generator holds the rule set to result-preservation, and the suite carries a second obligation this design introduces: a learned component can be entirely *inert* while every answer stays correct, so a test must assert that the learning happened and not only that the answer did.

Carbonite, the resource manager. Carbonite decides whether a plan is feasible, hands out memory reservations and shuffle credits, and decides when a query goes out of core; it never rewrites a plan and never computes a result, and that restriction dictates its interface. Conventional admission control answers a boolean, which is useless to a planner because it names no smaller thing to try, so Carbonite returns the binding constraint, the *operator* that binds, and a counter-offer Kyber can re-plan around — precisely because the layer that may rewrite the plan is not this one. Flow control is credit-based, one credit being one in-flight batch slot, so a channel’s window is a direct bound on its memory rather than an estimate of it, and the same transport carries the shuffle and the CPU- to-GPU hand-off of Section 5. Bulk data never enters the Ray object store, which is what keeps distribution a scheduling concern rather than a serialization one.

How Batcher compares. **DuckDB** [39] is the single-node bar: vectorized in the MonetDB/X100 lineage [8], a compressed native store with zone maps, optimized once before execution, and it does not distribute. **Spark** [4, 48] distributes, and its Adaptive Query Execution [42] re-plans on measured statistics at stage boundaries. **Ray Data** [30], on Ray [33], is a streaming batch runtime for ML pipelines with actor pools for GPU work and no relational optimizer; we benchmark against it throughout because a data engine claiming to serve AI workloads should be held to the standard of the runtime those workloads use today, which makes it a yardstick rather than a rival. **Daft** [17] and **Polars** [47] are native Rust engines over Arrow [43]. Batcher does not lead because its kernels are faster than everyone else’s; on several axes it is at parity, and on storage it is behind, because a compressed, dictionary-encoded, zone- mapped format is exactly what Section 2’s Arrow-only contract forecloses. It leads on the axis none of them occupies: every input to a planning decision — cardinality, cost coefficients, physical strategy, memory sizing — comes from a measured run rather than a shipped constant, and none of the five learns anything across queries at all. The sharpest departure from Spark is the unit of movement: Spark moves rows through JVM serialization, which was reasonable before an ecosystem agreed on a columnar format, where Batcher moves columnar batches with no serialization step, so the memory one operator wrote is the memory the next one reads. That is the largest single source of the gap in Table 2, and an ecosystem change more than an engineering one.

	sf1		sf10	
	b/x	wins	b/x	wins
Baseline				
DuckDB (native store)	0.79	16/22	1.29	8/22
DuckDB (same Arrow)*	0.26	22/22	—	—
Polars [47]	0.43	21/22	0.37	18/22
Daft [17] [†]	0.35	20/20	0.42	16/20
Spark [4] [‡]	0.041	22/22	0.074	22/22

Table 2: TPC-H, all 22 queries, correctness-gated, one 96-core node; *b/x* below 1.00 means Batcher is faster. Suite totals (ms), sf1 / sf10: Batcher 472 / 3,707; DuckDB 594 / 2,038; Polars 1,173 / 9,914; Daft 1,152 / 8,680; Spark 14,555 / 57,781. *DuckDB executing the same zero-copy Arrow Batcher is given, not a compressed store it was allowed to build first; not reported at sf10, because over Arrow views DuckDB’s planner has no storage statistics to order a many-way join with. [†]Daft covers the 20 queries it can express, and returns wrong answers on q6 and q15. [‡]Spark runs in local mode, which is not the deployment it is built for; its roughly 90 ms fixed per-query floor is most of the ratio at these scales, and no claim in this paper rests on the column.

7 Evaluation

Ratios follow the convention of Section 1.1 throughout. The evaluation tests the architectural claim before it tests speed, then asks whether the breadth costs anything on standard relational work, whether the residual losses are storage or execution, what a query *costs* as opposed to how long it took, and how the design behaves where standard suites do not look.

Setup. One Intel Xeon Platinum 8275CL at 3.00 GHz, 96 vCPU (48 physical), 184 GiB, Linux 6.8; distributed results on an 8-node Ray cluster, 128 vCPU total, whose per-node inbound bandwidth we report as measured rather than as a specification. The injected-failure arms use a single-box local Ray cluster, which removes the network entirely. GPU results are on 8×T4, two accelerator generations behind current hardware, so the CPU-to-GPU cost ratio those numbers assume is more favourable to CPU-side preprocessing than it would be on an H100. DuckDB 1.5.5, Polars 1.36.1, Daft 0.7.21, Ray 2.56.0, PySpark 4.2.0, PyArrow 19.0.1. The harness refuses to report a timing for any query whose result does not match DuckDB, compared as an order-independent multiset at a float tolerance of 1.5×10^{-6} absolute and 10^{-9} relative. Every engine receives the identical warm-up — one correctness run, one warm-up, five timed — and the operator mix fans one SQL string to every SQL engine including Batcher, so no hand-written Batcher pipeline races a comparator’s SQL. Timings are best-of-*N* warm, and each suite runs a two-engine lineup, because a four-engine run holding DuckDB’s tables, its Arrow views and Polars’ frames at once reads H2O `join` at $1.83\times$ against the $0.93\times$ a two-engine run reads: the lineup is part of the measurement. Two hazards produced a wrong number here before they were controlled for — a shared machine is not a benchmark machine, and an in-place rebuild of the compiled extension silently replaces the engine mid-benchmark — so the harness stamps the build profile and the extension’s timestamp around every run.

Suite	Queries	geomean <i>b/x</i>	wins
JSON (semi-structured)	5	0.25 $\times$	5/5
ClickBench [13] [†]	43	0.64 $\times$	28/43
Operator mix	19	0.65 $\times$	12/19
TPC-H [46] sf1	22	0.79 $\times$	16/22
H2O.ai <code>join</code>	5	0.93 $\times$	3/5
TPC-DS [45] sf1	98	0.96 $\times$	38/98
H2O.ai <code>groupby</code>	10	1.19 $\times$	4/10
TPC-H sf10	22	1.29 $\times$	8/22
Join Order [29]	109	1.29 $\times$	35/109

Table 3: Every relational suite in the harness against DuckDB’s *native compressed store* — the harder of the two bars — on one 96-core node, ordered by result; six of nine go to Batcher. On identical Arrow input, the like-for-like execution bar, *every suite that can run it is a win*: JSON $0.039\times$ (5/5), ClickBench $0.072\times$ (43/43), H2O `groupby` $0.089\times$ (10/10), H2O `join` $0.244\times$ (5/5), TPC-H $0.256\times$ (22/22), the operator mix $0.362\times$ (15/19). TPC-DS and JOB have no entry on that bar because DuckDB over Arrow views has no storage statistics to order a many-way join with. The operator-mix row is the controlled instrument discussed in Section 7.3: 19 single-operator cases, every engine on the same in-memory Arrow input, PyArrow (Acero) and Daft each unable to express some of them. [†]Five ClickBench queries are answered from a recorded statistic rather than executed; over the other 38 the figure is $0.77\times$, and two operator-mix cases are answered the same way, which moves its two DuckDB geomeans to 0.76 and 0.50 over 17 cases.

7.1 Does the algebra remove the fork?

The central claim is architectural, so most of its evidence is a testing argument rather than a timing. Equation (1) is property-tested over random partitionings, and an integration test asserts that single-node equals multi-partition. An operator matrix then crosses every execution mode with every value edge, because the defects that reach production here are a non-default flag on a non-default path: a descending sort returned unsorted data under spill, and a distributed `GROUP BY` on a float key split one group in two, and both passed every single-axis test.

Two measurements go further than the test suite, because a generator only ever produces the case the algebra was designed for. **The whole suite, through both schedulers:** every TPC-H query was executed twice on the same box, once through the single-node executor and once through the distributed one, and compared as row multisets. **All 22 agree.** Distribution charges a geometric-mean $28.3\times$ at sf1, which is the correct answer for a six-million-row table on one machine and why the engine’s `auto` policy declines it at this size. The claim is not that distribution is free; it is that it is a per-query decision whose price is visible, and that taking it does not change the answer. **And a partitioning nobody planned:** a worker dying mid-query leaves the surviving partials to be recombined in an order and a grouping no planner chose, which Section 7.7 injects six times with every recovered relation identical to the single-node answer. A system carrying a second distributed implementation would have to get that path right in that implementation too.

7.2 Relational breadth

Table 2 reports TPC-H and Table 3 every relational suite in the harness. Batcher leads DuckDB’s *compressed native store* at sf1 — a storage engine *and* an execution engine against an execution engine alone — and trails it at sf10. That boundary is not a uniform degradation, and locating it matters. Of thirteen shapes whose cost can be tracked across the two scales, nine cost *less* than ten times as much for ten times the rows; four are superlinear and carry the loss between them, and they are the benchmark’s two highest-cardinality group-bys (q13, q18), its largest intermediate (q9) and its six-way join (q5). An engine uniformly slower at scale would show no such split.

Correctness is not free for everyone: Daft returns a wrong TPC-H q6 at both scales, and Polars cannot parse eight of the 99 TPC-DS queries. Hence the gate on every timing, and the reporting of such a case as incomparable rather than as a win — a benchmark that does not check results reports the fastest wrong answer.

Three rows deserve a sentence each. **TPC-DS** registers all 99 queries rather than a curated subset, because a suite that registered only the queries the front-end already runs would report completeness and measure nothing. **The Join Order Benchmark** [29] is aimed squarely at this paper’s thesis and is the one Batcher loses: its queries run over a real IMDb snapshot whose predicates are correlated the way real data is, which is exactly where the independence assumptions in a textbook cost model fail, and Leis et al. built it to show that estimation error rather than the join-order search is where optimizers lose. Section 8 reports why the mechanism meant to close that gap is currently inert. **H2O.ai** [21] isolates what TPC-* buries inside larger plans, and the split is legible: Batcher takes the join task and loses the group-by, whose keys are low-cardinality strings DuckDB holds dictionary-encoded and groups on eight-bit codes where Batcher reads full Arrow `Utf8`. On identical Arrow input the same suite is $0.089\times$ and 10 of 10.

7.3 Storage or execution?

The operator mix of Table 3 holds everything but the one variable fixed: one input representation, one machine, no SQL front-end differences, no scan in the timed region, and DuckDB present twice differing *only* in whether it was allowed to ingest that identical Arrow input into its own compressed store first. Across its 19 cases Batcher takes 15 from DuckDB-on-Arrow and 12 from DuckDB’s native store — so of the seven cases the store wins and the execution engine does not, it wins by ingesting rather than by executing.

The four Batcher losses to DuckDB-on-Arrow are the informative ones, because they are the only cases where the input is genuinely equal and Batcher is still behind: three string sorts and a high-cardinality `DISTINCT`. Arrow’s `Utf8` keeps a string’s bytes in a separate buffer behind a 32-bit offset, so ordering one is a gather through an indirection where DuckDB compares a 16-byte value with its first bytes inline — the German-string representation Umbra introduced [37]. That is a data-layout gap in the columnar format rather than in either engine, and it is the gap the H2O group-by result attributes

to storage. It is measurable directly: grouping a dictionary column is about $7\times$ faster than grouping the decoded one, so handing Batcher DuckDB’s own representation across the FFI makes it *slower*, because the boundary decodes it to satisfy the Arrow-only contract. A change that would have let a dictionary reach the kernel was built, measured and reverted for regressing plainer shapes. This is the one place where the paper’s central constraint is paid for in performance rather than repaid.

On ClickBench [13] the pattern holds at suite scale: Batcher wins 43 of 43 against DuckDB on identical Arrow input, and wins the suite against the native store too, more narrowly. One variable, isolated, and the residue is storage, not kernels.

7.4 What a query costs, as opposed to how long it took

Every ratio so far is wall clock, and wall clock answers only how long you waited. It says nothing about what the answer cost, which on rented hardware is the question. An engine that spreads a query over four times the cores finishes sooner and bills four times as much, and no comparison in this literature reports it. It is measurable here for a specific reason: Batcher and DuckDB both execute in-process, so `getrusage` charges both engines’ threads to one counter and the delta across a call is that engine’s CPU time. On TPC-H sf1, all 22 queries, gated on correctness before timing: **Batcher buys a $1.3\times$ wall-clock win with $1.9\times$ the CPU, by running on $2.4\times$ the cores** — geometric-mean $0.78\times$ wall against $1.86\times$ CPU, at 21.9 effective cores against DuckDB’s 9.2, excluding the one query answered from a recorded statistic rather than executed. The wall figure reproduces Table 2’s 0.79, which cross-checks that this is the same engine.

We report it because it is the honest shape of the result and because it bounds the claim. On a machine already bought, spending idle cores to return sooner is free and the wall-clock ratio is the whole story; on a machine rented by the core-hour it is not, and a reader optimizing spend should conclude that Batcher is the wrong choice for a batch job with no latency requirement. Neither engine saturates this box — DuckDB never exceeds ten effective cores of ninety-six — so the gap is as much about how much parallelism each engine extracts as about waste.

The small case, which a warm benchmark never measures.

Section 2’s first consequence is that a small query pays no distribution tax, and a suite reporting best-of- N warm timings structurally cannot check it, because such a suite never sees a shape for the first time — which is what a user issues constantly. Measured separately, with a fresh literal per iteration so the plan cache cannot hit, optimizing a never-seen filter $\rightarrow$ join $\rightarrow$ group-by costs 2.75 ms, and end to end over twelve never-seen shapes at $n = 10^4$ Batcher runs at $0.80\times$ DuckDB on a cold filter and $0.27\times$ on a cold `count(*)`. DuckDB pays a first-query cost too, and ours is the smaller, which is where a Python control plane could have been disqualifying and is not. What remains is a *warm* gap on the narrowest shape, 4.07 ms against 2.04 ms, of which planning is 0.002 ms — per-query fixed cost at the FFI boundary, not the optimizer.

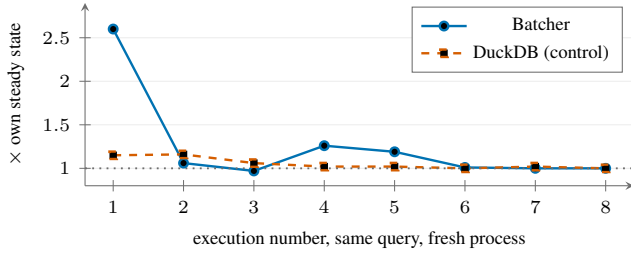


Figure 5: The learning loop, with the control that rules out the obvious alternative explanation. Each TPC-H query runs eight times back to back in a fresh process (the learned store is per-process, so a new process is a cold hub), normalized to that engine’s own steady state; the curve is the geometric mean over 22 queries. DuckDB runs the identical loop on the identical data and has no cross-query learning, so its curve is the environmental floor — page cache, JIT, allocator. Batcher’s first execution costs $2.6\times$ steady state where the floor is $1.1\times$, and the excess is gone by the second or third execution. Runs 4–5 reading 1.2 is box noise, not a regression; a second six-run round read 2.69 .

7.5 Streaming, against a streaming engine

Two measurements test the shared-operator claim from opposite directions. **Against a streaming engine**, draining a four-million-row Parquet backlog through a grouped aggregation, correctness-gated, Batcher runs at 18.9M rows/s against Spark Structured Streaming’s [3] 6.0M — a $3.2\times$ lead — under the memory bound flat in the input reported in Section 2.3. **Against a batch engine**, DuckDB answers that same aggregation in 15.9ms as a one-shot query, $13\times$ faster than Batcher’s streaming path. That gap is not the operators, which are shared and are the paper’s claim; it is the streaming *driver*, which plans a micro-batch, commits offsets and writes a checkpoint per trigger while a batch query does none of those things. What this supports is that the operators are one implementation and the memory bound holds for both, not that a streaming query is as cheap as a batch one. It is not, by an order of magnitude, and the cost is the driver.

7.6 Does the engine actually get faster the second time?

The cross-query learning loop is the capability no comparator here has, and without a control a before-and-after number would be worthless, because “the second execution is faster” is also what a page cache, a JIT and an allocator arena produce. Figure 5 measures the whole curve against that control: DuckDB supplies the floor at $1.1\times$ on its first execution and flat after, while Batcher pays $2.6\times$ and recovers it within two executions, so roughly a $2.4\times$ first-execution premium is attributable to something the control does not have. One honest limit: the first-to-second transition folds the *plan cache* together with the *learned store*, and this experiment does not separate them. It establishes that the loop exists and what it is worth end to end, not which half did the work.

Arm	ms	vs. no fault	Result
no fault, replication = 1	258.0	$1.00\times$	identical
no fault, replication = 2	172.7	$0.67\times$	identical
1 lost at reduce, replication = 1	733.6	$2.84\times$	identical
1 lost at reduce, replication = 2	2167.1	$8.40\times$	identical
2 lost at reduce, replication = 2	2215.4	$8.59\times$	identical
1 lost during map	1723.1	$6.68\times$	identical
2 lost during map	1755.7	$6.81\times$	identical

Table 4: Injected worker loss on a grouped aggregate, 8M rows over $5,000$ keys, 8 workers, best of three. A worker lost *at reduce* has already published its buckets, so they vanish; one lost *during map* has published nothing, so its source split is relocated. The last column is load-bearing: a lost bucket reads back as an *empty* bucket rather than as an error, so a defect here would drop rows silently. Absolute times are from a single-box local Ray cluster.

7.7 Scaling out, and what a lost worker costs

Aggregate all-to-all shuffle throughput on an 8-node Ray cluster is 2.0GB/s at two worker nodes, 6.9 at four and 15.2 at eight — superlinear, $7.6\times$ for $4\times$ the nodes. The mechanism is the algebra: mappers publish pre-aggregated state rather than rows, so adding a node adds reduce capacity instead of contention, and a combiner tree plus credit-based flow control keeps per-node memory bounded as the cluster widens. Two qualifications belong with that number rather than after it. A *single* reducer stays pinned at its NIC ($\sim 2.7\text{GB/s}$), so the gain is in the aggregate and not in any one channel; and at two nodes the shuffle does not saturate the available reduce capacity, so part of the 2-to-4 gain is recovered inefficiency rather than scaling. The shape is what we stand behind — nodes add throughput at no per-node memory cost — not $1.9\times$ per doubling forever. Distribution is also not always the right answer, and the engine says so: forcing `distributed=True` on in-process data costs $1.9\text{--}4.5\text{s}$ a query, which is exactly the tax Section 2 exists to make optional.

A distributed claim that is only a throughput claim is half a claim, because the property a shuffle’s design exists for is what happens when a worker dies. Table 4 injects the failure directly. Six arms, two failure phases, up to two simultaneous losses, and every recovered relation identical to the single-node answer — the assertion Equation (1) is for, tested against real failures rather than random partitionings. Recovery is also parallel across losses, since two workers lost costs what one costs, which falls out of the algebra too: what is regenerated is a partial, and partials regenerate independently. Replication, however, makes recovery worse. Copying every published bucket to a survivor is meant to convert a recompute into a re-fetch; instead it turns a $2.84\times$ recompute into an $8.40\times$ recovery. The environment is a confound — replication’s premise is that re-reading the source from object storage dominates, and on a single box with an in-memory source that premise is false — so the measurement does not refute the design. It establishes that the benefit is unproven in every environment this system has been measured in, and that the fixed cost is real.

7.8 GPU and ML workloads

These are the workloads Ray Data was built for, which makes it the right yardstick and the wrong thing to keep score against. The question is whether an engine that treats decode and inference as relational operators can hold its own against a runtime purpose-built for them. It can: on 8×T4 with real models, ResNet-50 batch inference 2.05×, image embeddings 1.98×, fractional-GPU packing 1.96×, video-clip inference 3.6× and training-data ingest 3.0× (Figure 1, right). Three figures are much larger — audio feature extraction 12.5×, LLM batch inference 11.1×, text embeddings 47× — and should not be quoted as general execution speed, because they come from scheduling rather than kernels and are choices any runtime could adopt. The durable claim is the 2–3.6× range: putting AI operators inside a query engine costs nothing against a specialized runtime. One exception bounds it — on a maximally-large compute-bound job both engines saturate the same GPU at the same FLOPs, so the ratio is parity. These ratios are also only meaningful if the comparator is configured properly, and ours initially was not: our adapter registered each table as a single block, which is Ray Data’s unit of parallelism, so a 6M-row table ran on one core of 96. That was our defect, and every number above is from the corrected adapter.

8 Limitations and threats to validity

Breadth invites the suspicion that something is being averaged away, so this section names what is not won, what the numbers above cannot see, and where an honest reader should discount them.

Storage is the largest open gap, and it is structural. Batcher owns no storage format: Arrow is the only columnar representation any operator sees, which forecloses a compressed, dictionary-encoded, zone-mapped store and a German-string layout [37] in one stroke. Three results above are that decision’s bill — TPC-H at sf10 (1.29×), the H2O group-by (1.19×), and the string sorts that lose even on identical input — and closing it needs a plan’s logical type separated from a morsel’s physical one, an architectural change rather than a kernel one. The scan path is a second, independent term: Batcher’s Parquet decode costs 1.7–3.0× DuckDB’s CPU, which places it in the Arrow Rust decoder rather than in anything specific to this engine.

The learned loop is not yet answering the benchmark built for it. The Join Order Benchmark remains 1.29× behind, and the mechanism this paper argues should close it is inert on the shape that matters: the key a measured cardinality is filed under and the key the estimator reads back disagree for the commonest aggregate shape, so the loop writes a true row count that nothing consumes. The instructive part is why it cannot be fixed in isolation. Correcting the key alone makes the engine *slower* — TPC-DS moves 0.965 → 0.988 and JOB 1.252 → 1.293 — because every threshold downstream of that input was calibrated while it was an order of magnitude low. A learning loop

is a closed system, and repairing one edge is a recalibration rather than a patch; it is pinned as a failing test, not described as done. A second limit sits on the positive result: the 2.4× first-execution premium of Figure 5 folds the plan cache together with the learned store.

Streaming is micro-batch, and does not reach Flink’s guarantees. There is no operator graph, no barrier protocol and no keyed state backend; the driver is a Python loop over pull, execute, write, and a single point of failure with no standby. Watermarks do not propagate across operators because there is no graph to propagate along, and streaming state is one in-memory Arrow batch that does not spill. The driver is also expensive in absolute terms (Section 7.5), so the shared-operator claim buys maintainability and a memory bound, not streaming performance competitive with a one-shot query. Exactly-once holds for Delta, for a file sink by batch position, and for a *keyed* database write; an *append* to Iceberg or Hudi writes its rows twice on a replayed epoch.

Recovery is correct; replication does not pay for itself. Correctness under worker loss holds across every arm tested, which is the property the algebra is for; the performance half does not, and the one environment we can measure it in is also the one most hostile to its premise. Replicas are resident rather than on a durable tier, and Batcher has no equivalent of Spark’s external shuffle service, so a whole-node loss that takes every copy still costs a recompute round. Two operator-level ceilings stand as well: the CPU-to-GPU split happens at exactly one resource boundary, so a three-stage chain rides its third stage in the consumer, and the inequality join is not distributed at all (Section 2.2). Finally, Batcher owns no storage, so there is no transaction manager, no MVCC and no index — an engine whose floor is milliseconds is not a read path an application points transactions at.

Wall clock is not the only currency. Section 7.4 measures rather than flags it: 0.78× the wall time for 1.86× the CPU, on 2.4× the cores. On hardware already bought that is a pure win, because the cores were idle; on hardware rented by the core-hour it is a loss for any batch job with no latency requirement, and a reader in that position should choose the other engine. We report both and do not net them together.

Threats to validity. In descending order of how much they should move a reader. *One machine*, and one of them for the distributed results too: the sf1-to- sf10 attribution is a two-point scaling study and not a scaling law, and the worker-loss arms run on a single-box local Ray cluster that removes the network replication’s premise rests on. *Warm, best-of-N*: the headline suites report the best of several runs of the same query, which is exactly the shape a memoizing engine wins, and Batcher’s session carries a plan cache and a learned-stats loop across those repeats where DuckDB and Polars have no counterpart. *The comparator pays a conversion Batcher does not*: every engine is asked for a `pyarrow.Table`, and on a query returning nine million rows that conversion is roughly *half* of DuckDB’s

measured time. *The gate compares multisets*, and was blind to whether a sort had happened at all until the ordered cases were given an ordered comparison. *Comparator configuration*: we found and fixed one adapter defect that cost Ray Data 95 of 96 cores, and three in the Spark adapter. *Process*: continuous integration installs neither Ray nor a GPU, so the distributed and GPU results rest on recorded runs rather than on the gate that runs on every commit, and are the ones a reader should trust least.

9 Related work

Batcher sits at the intersection of five lines of work, and takes something from each.

Vectorized and compiled single-node execution. The columnar, batch-at-a-time model begins with MonetDB/X100 [8] and the compiled alternative with HyPer [36]; Umbra [37] contributed the inlined-prefix string representation that Section 8 names as Batcher’s largest structural gap. Batcher’s execution is morsel-driven [28] and vectorized, with a Cranelift [11] compiled path carrying a proven-identical fallback, and DuckDB [39] is the closest single-node comparator. Nearest to Batcher’s *implementation* choices are the embeddable Arrow-native engines — Velox [38], Photon [7] and DataFusion [27] — which share the columnar contract and the library-rather-than-server posture. None is distributed on its own, none carries a learned cross-query loop, and none treats media decode or inference as an expression: a Velox or Photon plan is executed by a host system supplying exactly the layer Section 2 argues should not be separate.

Distributed dataflow and cloud warehouses. MapReduce’s combiner [16] and Gray et al.’s algebraic aggregate [20] are where the mergeable triple comes from; Batcher’s contribution is not the triple but the refusal to admit an operator without one. Spark [4, 48] is the direct ancestor of the design Section 6 departs from, the cloud warehouses [15, 41] answer the same scale question with a service rather than a library, and the lake-house line [2] is where Batcher’s table formats come from. Volcano [19] established the operator/exchange separation this paper leans on, and our departure is that there is no distributed operator to separate.

Streaming and unified execution. Structured Streaming’s framing of a stream as an unbounded table [3] is the framing Section 2.3 adopts. The two closest ancestors of this paper’s central device are here rather than in the operator literature: Summingbird [9] made the monoid the explicit unifying abstraction so that one program compiles to both a batch and an online engine, and the Dataflow model [1] generalized the two into a single specification. The difference we claim over both is not the algebra but the level: they unify the program or the specification above two engines, and we make mergeability the admission test of one engine’s own operators. Flink [12] and Naiad [35] are the dataflow answer, with the operator graph,

barrier protocol and keyed state backend Batcher does not have and does not claim, and DBSP [10] derives incremental operators from a query algebra — close in spirit to reusing the triple as the incremental form, and more general.

Adaptive and learned optimization. Estimation begins with Selinger [40]; mid-query re-optimization and eddies [6] are the lineage of the stage- boundary clock, with Spark AQE [42] its closest deployed relative — the same mechanism and granularity, which this paper is careful not to overclaim past. Learned optimizers [31, 32] attack estimation with models that change which plan runs; Batcher takes the lighter route of sketches [14, 18, 24], refit cost coefficients and a bandit [5] over *result-equivalent* strategies, which buys reproducibility and a cold start that is exactly the untuned system, and gives up the ability to correct a genuinely bad plan choice. Leis et al. [29] is both the standing argument for measured cardinalities and the benchmark Batcher still loses.

Data systems for machine learning. Ray [33] and Ray Data’s streaming batch model [30] are the runtime the AI evaluation is measured against and the closest system to Section 5; Daft [17] is the closest multimodal relational engine, and MosaicML’s StreamingDataset [34] is where the computed-permutation epoch order comes from. Putting a model inside the query optimizer is itself established: NoScope [23] and BlazeIt [22] optimize queries over neural-network inference with cascades, proxy models and control variates. The difference is scope rather than ambition — those systems optimize *around* inference for a class of query, approximating where the semantics permit it, while Batcher makes decode, embedding and inference ordinary expressions in the relational algebra, exact by construction. Their techniques are strictly more powerful on the queries they target and strictly less general.

10 Conclusion

The workarounds for the shifts of Section 1 read as the cost of doing business: a warm cluster so small queries are bearable, a second execution model for anything fresher than a day, a shuffle width tuned by hand, a per-row callback for whatever SQL cannot say, and a duplicate of the data for whoever trains the models. Our claim is that none of it followed from a law. Each fork was the right local answer to a constraint that has since inverted, and admitting an operator only as a mergeable triple retires four at once: the split between the fast engine and the scalable one; the split between batch and streaming, because the triple is the incremental form; the split between planning and measuring, because an operator identical everywhere makes a measurement taken anywhere valid everywhere; and the split between the query and the model, because decode, embedding and inference are expressions in the same algebra over the same batches. The breadth is not four features that were built. It is one constraint that was not broken.

The constraint is not free, and the paper is most useful where it says so. It refuses exact order statistics, leaves the inequality

join undistributed, costs $1.86\times$ the CPU of the engine it beats on the clock, and its replication currently makes recovery three times more expensive rather than cheaper. Each is stated in Section 8 with a number beside it.

What the constraint does not buy is a storage format, and that is where the last single- node gap lives. Give DuckDB the same Arrow bytes Batcher executes over and it loses every suite that comparison can run, by 2.8 to $26\times$; give it back its own compressed, dictionary-encoded, zone-mapped store and at ten times the data it wins. Both facts are the same fact. The incumbent’s remaining advantage on a single node is a file format, not an engine — a far more portable thing to lose to — and the next version of this system is a plan’s logical type separated from a morsel’s physical one.

Reproducibility. Every number comes from a committed harness, and each experiment is one named script: `benchmarks/run.py` drives the engine adapters and `harness.py` enforces the correctness gate, with `scenarios/cost_bench.py`, `cluster/robustness/worker_loss.py`, `scenarios/streaming_throughput.py`, `scenarios/learning_curve.py` and `scenarios/scheduler_equivalence.py` producing Section 7.4, Table 4, Section 7.5, Figure 5 and the two-scheduler agreement of Section 7.1 respectively. The harness records the engine build profile with each run, so a result cannot silently be attributed to a different build.

References

- [1] T. Akidau, R. Bradshaw, C. Chambers, S. Chernyak, R. J. Fernández-Moctezuma, R. Lax, S. McVeety, D. Mills, F. Perry, E. Schmidt, and S. Whittle. The dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. *Proceedings of the VLDB Endowment*, 8(12):1792–1803, 2015.
- [2] M. Armbrust, T. Das, S. Paranjpye, et al. Delta Lake: High-performance ACID table storage over cloud object stores. *Proceedings of the VLDB Endowment*, 13(12), 2020.
- [3] M. Armbrust, T. Das, J. Torres, B. Yavuz, S. Zhu, R. Xin, A. Ghodsi, I. Stoica, and M. Zaharia. Structured streaming: A declarative API for real-time applications in Apache Spark. In *ACM SIGMOD International Conference on Management of Data*, 2018.
- [4] M. Armbrust, R. S. Xin, C. Lian, Y. Huai, D. Liu, J. K. Bradley, X. Meng, T. Kaftan, M. J. Franklin, A. Ghodsi, and M. Zaharia. Spark SQL: Relational data processing in Spark. In *ACM SIGMOD International Conference on Management of Data*, 2015.
- [5] P. Auer, N. Cesa-Bianchi, and P. Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256, 2002.
- [6] R. Avnur and J. M. Hellerstein. Eddies: Continuously adaptive query processing. In *ACM SIGMOD International Conference on Management of Data*, 2000.
- [7] A. Behm, S. Palkar, U. Agarwal, et al. Photon: A fast query engine for lakehouse systems. In *ACM SIGMOD International Conference on Management of Data*, 2022.
- [8] P. A. Boncz, M. Zukowski, and N. Nes. MonetDB/X100: Hyper-pipelining query execution. In *Conference on Innovative Data Systems Research (CIDR)*, 2005.
- [9] O. Boykin, S. Ritchie, I. O’Connell, and J. Lin. Summingbird: A framework for integrating batch and online MapReduce computations. *Proceedings of the VLDB Endowment*, 7(13), 2014.
- [10] M. Budi, F. McSherry, L. Ryzhyk, and V. Tannen. DBSP: Automatic incremental view maintenance for rich query languages. *Proceedings of the VLDB Endowment*, 16(7), 2023.
- [11] Bytecode Alliance. Cranelift code generator, 2024.
- [12] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas. Apache Flink: Stream and batch processing in a single engine. *IEEE Data Engineering Bulletin*, 38(4), 2015.
- [13] ClickHouse, Inc. ClickBench: A benchmark for analytical databases, 2024.
- [14] G. Cormode and S. Muthukrishnan. An improved data stream summary: The count-min sketch and its applications. *Journal of Algorithms*, 55(1):58–75, 2005.
- [15] B. Dageville, T. Cruanes, M. Zukowski, et al. The Snowflake elastic data warehouse. In *ACM SIGMOD International Conference on Management of Data*, 2016.
- [16] J. Dean and S. Ghemawat. MapReduce: Simplified data processing on large clusters. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2004.
- [17] Eventual Computing. Daft: A distributed query engine for large-scale multimodal data, 2024.
- [18] P. Flajolet, É. Fusy, O. Gandouet, and F. Meunier. HyperLogLog: The analysis of a near-optimal cardinality estimation algorithm. In *Conference on Analysis of Algorithms (AoFA)*, 2007.
- [19] G. Graefe. Volcano—an extensible and parallel query evaluation system. *IEEE Transactions on Knowledge and Data Engineering*, 6(1):120–135, 1994.
- [20] J. Gray, S. Chaudhuri, A. Bosworth, A. Layman, D. Reichart, M. Venkatrao, F. Pellow, and H. Pirahesh. Data cube: A relational aggregation operator generalizing group-by, cross-tab, and sub-totals. *Data Mining and Knowledge Discovery*, 1(1):29–53, 1997.
- [21] H2O.ai. db-benchmark: Database-like ops benchmark, 2023.
- [22] D. Kang, P. Bailis, and M. Zaharia. BlazeIt: Optimizing declarative aggregation and limit queries for neural network-based video analytics. *Proceedings of the VLDB Endowment*, 13(4), 2020.
- [23] D. Kang, J. Emmons, F. Abuzaid, P. Bailis, and M. Zaharia. NoScope: Optimizing neural network queries over video at scale. *Proceedings of the VLDB Endowment*, 10(11), 2017.
- [24] Z. Karnin, K. Lang, and E. Liberty. Optimal quantile approximation in streams. In *IEEE Symposium on Foundations of Computer Science (FOCS)*, 2016.
- [25] Z. Khayyat, W. Lucia, M. Singh, M. Ouzzani, P. Papotti, J.-A. Quiané-Ruiz, N. Tang, and P. Kalnis. Lightning fast and space efficient inequality joins. *Proceedings of the VLDB Endowment*, 8(13), 2015.
- [26] A. Kipf, T. Kipf, B. Radke, V. Leis, P. Boncz, and A. Kemper. Learned cardinalities: Estimating correlated joins with deep learning. In *Conference on Innovative Data Systems Research (CIDR)*, 2019.
- [27] A. Lamb, Y. Shen, et al. Apache Arrow DataFusion: A fast, embeddable, modular analytic query engine. In *ACM SIGMOD International Conference on Management of Data*, 2024.
- [28] V. Leis, P. Boncz, A. Kemper, and T. Neumann. Morsel-driven parallelism: A NUMA-aware query evaluation framework for the many-core age. In *ACM SIGMOD International Conference on Management of Data*, 2014.
- [29] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann. How good are query optimizers, really? *Proceedings of the VLDB Endowment*, 9(3):204–215, 2015.
- [30] F. S. Luan, R. Y. Wang, Y. Gu, Z. Mao, C. Lin, A. Kamsetty, H. Chen, C. Su, B. Veeramani, S. Lee, S. Cho, C. Zinzow, E. Liang, I. Stoica, and S. Wang. The streaming batch model for efficient and fault-tolerant heterogeneous execution. arXiv:2501.12407, 2025.
- [31] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, and T. Kraska. Bao: Making learned query optimization practical. In *ACM SIGMOD International Conference on Management of Data*, 2021.
- [32] R. Marcus, P. Negi, H. Mao, C. Zhang, M. Alizadeh, T. Kraska, O. Papaemmanouil, and N. Tatbul. Neo: A learned query optimizer. *Proceedings of the VLDB Endowment*, 12(11), 2019.
- [33] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan, and I. Stoica. Ray: A distributed framework for emerging AI applications. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.
- [34] MosaicML / Databricks. StreamingDataset: Fast, accurate streaming of training data from cloud storage, 2023.
- [35] D. G. Murray, F. McSherry, R. Isaacs, M. Isard, P. Barham, and M. Abadi. Naiad: A timely dataflow system. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2013.
- [36] T. Neumann. Efficiently compiling efficient query plans for modern hardware. *Proceedings of the VLDB Endowment*, 4(9):539–550, 2011.
- [37] T. Neumann and M. Freitag. Umbra: A disk-based system with in-memory performance. In *Conference on Innovative Data Systems Research (CIDR)*, 2020.
- [38] P. Pedreira, O. Erling, M. Basmanova, et al. Velox: Meta’s unified execution engine. *Proceedings of the VLDB Endowment*, 15(12), 2022.
- [39] M. Raasveldt and H. Mühleisen. DuckDB: An embeddable analytical database. In *ACM SIGMOD International Conference on Management of Data*, 2019. Demonstration.
- [40] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. Access path selection in a relational database management system. In *ACM SIGMOD International Conference on Management of Data*, 1979.
- [41] R. Sethi, M. Traverso, D. Sundstrom, et al. Presto: SQL on everything. In *IEEE International Conference on Data Engineering (ICDE)*, 2019.
- [42] The Apache Software Foundation. Adaptive query execution — Spark SQL performance tuning guide, 2024.
- [43] The Apache Software Foundation. Apache Arrow: A cross-language development platform for in-memory analytics, 2024.
- [44] The Apache Software Foundation. Arrow Flight RPC, 2024.
- [45] Transaction Processing Performance Council. TPC Benchmark DS standard specification, 2021.
- [46] Transaction Processing Performance Council. TPC Benchmark H standard specification, 2022.
- [47] R. Vink and Polars contributors. Polars: DataFrames for the new era, 2024.
- [48] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2012.